On the Design, Implementation, and Execution Environment of the Typed AI Agent Language AIPL

Yasushi Kodama
Hosei University yass@hosei.ac.jp

2026-06-12

Abstract

AI systems built around large language models (LLMs) are rapidly converging on architectures in which multiple autonomous agents—planners, solvers, reviewers, tool runners—cooperate asynchronously by exchanging messages. This paper presents, in a single coherent account, the design, implementation, and execution environment of **AIPL** (Actor-based Intelligent Parallel Language), a typed AI agent language that expresses this structure as a first-class language construct. AIPL inherits the lineage of Hewitt's actor model (1973) and ABCL/1 (1986), onto which it layers (i) **Hindley–Milner (HM) type inference** that demands no annotations, (ii) **gradual typing features** such as capability effects, linear types and session types, and (iii) **first-class AI integration** that embeds LLM calls as the language primitive `ai_call`. On the implementation side, a single `.abcl` source is served by **seven runtimes** (annotated/inferred Python, the canonical OCaml, browser/Node JavaScript, and C) and **nine code-generation backends** (C+pthread, SDL2, Xinu, Python, Pony, Erlang, Go, Prolog, LLVM/OpenMP). We empirically verify that the three HM-equipped implementations (OCaml, inferred Python, and the C code generator) produce *identical inferred types* for the same source. For execution, the same source can be deployed across three concurrency models—1:1 OS threads, M:N lightweight threads, and a cooperative event loop—and we add a distributed runtime layer (DR-1..13) providing token-budget governance, checkpointing, quarantine, CRDT state and saga compensation, together with on-device distributed execution that spans a Mac and the bare-metal OS Xinu running on a Raspberry Pi. As evaluation we report cross-validation of type inference, a locality analysis of type soundness, and more than 213 green smoke tests/demos.

Contents

1 Introduction

1.1 Background and Motivation

As LLM capabilities improve, practical AI systems increasingly rely not on a single model call but on multiple agents with distinct roles (planner, solver, reviewer, tool executor) that *cooperate asynchronously through messages*. Strikingly, this is structurally isomorphic to the vision of **ABCL/1** [2], the object-oriented concurrent computation model proposed by Yonezawa et al. in 1986, in which independent stateful objects coordinate through three forms of message passing (past, now, future). However, the actor languages of that era lacked any facility for absorbing, at the language level, the *token-budget management, timeouts, providers, fallbacks and cross-machine coordination* that modern AI agents require.

This work designs and realizes **AIPL** (Actor-based Intelligent Parallel Language) as a language that fills this gap. Building on a direct re-implementation of ABCL, AIPL aims to:

1. realize an **annotation-free actor language** via Hindley–Milner type inference;
2. make LLM invocation a first-class primitive, `ai_call(provider, prompt)`; and

3. drive, from one source, three concurrency models (OS threads, lightweight threads, event loop) and beyond, up to distributed and on-device execution.

AIPL is therefore not merely a processor but a "scripting layer for an AI-OS"—a language layer through which the runtime governs the concurrency, persistence, budgets and cross-machine coordination of many AI agents.

1.2 Contributions

This paper makes four contributions.

(C1) An annotation-free actor language
We design an actor language that requires no type annotations in its surface syntax, inferring field types and method signatures from call sites and initializers, and we empirically verify that three independent HM implementations produce *identical inference results* for the same source (§9).

(C2) HM-driven code specialization
Using inference results, we establish a code-generation technique that *specializes* regions outside actor message boundaries from a `value_t` tagged union to native C types (§5.3).

(C3) First-class AI integration
An LLM can be driven from any of ABCL/1's three send forms, and the runtime governs token budgets, concurrency, fallback and usage accounting (§6.2).

(C4) One source, many concurrency models, distributed execution
The same `.abcl` source is deployed unmodified across three concurrency models, and we further demonstrate on-device distributed execution spanning a Mac and the bare-metal OS Xinu on a Raspberry Pi over HTTP/JSON and UART/TCP (§6,§7).

1.3 Organization

§2 reviews background and related work, §3 the language design, and §4 the type system and inference. §5 covers the realization of seven runtimes and nine backends; §6–§7 cover the execution environment (concurrency models, LLM integration, distributed runtime, evolutionary pipeline); §8 gives case studies; §9 evaluation, §10 discussion and limitations, and §11 concludes.

2 Background and Related Work

The actor model and ABCL. The actor model originates with Hewitt [1], treating each actor as a unit of computation that (i) receives a message, (ii) updates internal state, and (iii) creates new actors and sends. ABCL/1 [2] by Yonezawa et al. introduced three send forms—*past* (fire-and-forget), *now* (synchronous) and *future* (asynchronous handle)—and became a classic of object-oriented concurrent computing. AIPL inherits these three send forms directly (§3).

Hindley–Milner type inference. The HM type system, beginning with Hindley [3] and Milner [4], is guaranteed by Damas–Milner's Algorithm W [5] to infer principal types *without annotations*. AIPL's inferencer adopts a textbook HM comprising `unify` / `generalize` / `instantiate` and let polymorphism (§4.2).

Gradual typing, effects, linearity, sessions. Siek–Taha's gradual typing [6] enables a continuous coexistence of static and dynamic typing. While AIPL is centered on its inferred dialect, the annotated dialect retains capability effects [7], linear types [8] and session types [9] as a reference implementation, integrating the two as the inferred-HM axis and the annotated-gradual axis of one language (§4.3).

Structured concurrency and distribution. Structured concurrency via `scope { …}` shares its semantics with Java Project Loom's StructuredTaskScope [10]. At the distributed layer, AIPL incorporates CRDT-based [11] actor state replication and saga-style compensating transactions.

Evolutionary language design. AIPL's next-generation feature set was determined through a unique loop (§7.4): an evolutionary search based on MAP-Elites [12] empirically derives axes of language design, and the winners are implemented as features.

Comparison with existing actor/agent platforms. Erlang/OTP, Akka, Pony and Ray are mature actor/distributed platforms, but they pursue different design goals with respect to LLM integration, type inference and backend diversity (Table 1). AIPL's distinctiveness lies in simultaneously achieving (i) annotation- free HM inference over an actor language, (ii) projection from one frontend to nine backends, and (iii) first-class governance of LLM calls and token budgets at the language/runtime level.

Table 1: Comparison of design goals with existing platforms

	3 sends	inference	multi-backend	LLM first-class	main concurrency
Erlang/OTP	△	×	×	×	M:N (BEAM)
Akka (Scala)	△	△	×	×	M:N (JVM)
Pony	×	△	×	×	M:N (ref. caps)
Ray (Python)	△	×	×	△	1:1 + cluster
AIPL	✓	✓	✓	✓	1:1 / M:N / event-loop

3 The Design of AIPL

3.1 Computation Model

An AIPL program is a set of `class` declarations. Each `class` defines one actor species, and `new C(args)` creates an instance (actor). Each actor owns an independent **mailbox**, and its fields (state) are closed within the actor and cannot be read or written directly from outside. The sole interaction between actors is message passing.

3.2 The Three Send Forms

Inheriting ABCL/1, AIPL provides three send forms (Listing 1).

Listing 1: Three send forms and reply

```
class Counter {
  var count = 0;
  method tick() { count = count + 1; print("count = " + count); }
  method get()  { reply(count); }     // respond to a synchronous send
}
var c = new Counter();
send   c.tick();      // past   : fire-and-forget
var v = now c.get();  // now    : blocks until reply arrives
var f = future c.get();   // future : asynchronous handle
var w = await(f);         // obtain the value from the handle
```

`reply(value)` fulfills the reply slot of a now/future send. `future_done(f)` polls non-blockingly, and `await(f)` blocks to obtain the value. Within a method body, the special variables `self` (the actor) and `sender` (the sender) are available.

3.3 become and select

`become C(args)` replaces an actor's class and fields at runtime (a representation of state transition). `select { case …}` performs selective reception from multiple channels/messages, corresponding to Erlang's `receive`.

3.4 Channels and Structured Concurrency

`channel[T]` / `channel[T,cap=N]` provide CSP-style typed channels (`channel_send` / `channel_recv` / `select_recv`). A `scope { …}` block is a structured- concurrency construct that automatically joins the futures created within it upon exit, guaranteeing that the parent scope owns the lifetimes of its child tasks.

3.5 The LLM Integration Primitive

LLM invocation is built into the language as `ai_call([provider,] prompt)`. The provider id is `1`=Gemini (default) / `2`=Anthropic (Claude) / `3`=OpenAI / `0`=auto, and `AIPL_AI_PROVIDER=mock` takes all calls offline. An LLM can be driven from any of the three send forms (Listing 2).

Listing 2: Parallel queries via LLM × future

```
class Asker { method ask(q) { var r = ai_call(2, q); reply(r); } }
var a = new Asker();
var f1 = future a.ask("Translate hello to French.");
var f2 = future a.ask("Translate hello to German.");
print(await(f1));   // the two LLM calls run in parallel
print(await(f2));
```

4 Type System and Type Inference

4.1 The World of Types

AIPL's types are as follows. Base types: `int` (64-bit), `float` (double precision; a trailing dot is allowed, e.g. `100.`), `string` (UTF-8), `bool`, `unit`; structured types: `actor(C)` (an instance of class C), `array`, `tuple(…)` (positional, immutable, fixed arity; the length and each slot type are part of the type), `record{a:int,b:string}` (structural, named fields), `channel[T]`, function types, and `future`; plus implicit polymorphism via single-letter type variables (`T U V …`). A generic function can be written `function id(x:T)->T`.

4.2 Hindley–Milner Type Inference

The inferencer `infer.ml` of the canonical implementation (OCaml) realizes textbook HM: type variables via mutable union-find, `unify` / `generalize` / `instantiate`, and let polymorphism. The methods of a class are **pre-registered** with fresh type variables and concretized bottom-up during body checking—a **two-pass scheme**. As a result, information from call sites *automatically monomorphizes* method signatures.

For Listing 3, the inferencer derives `count : float`, `init : (int) -> unit`, `greet : () -> unit`. A method such as `Counter.dec(x)` is $\forall\alpha.\,\alpha \to \mathit{unit}$ at declaration, but is fixed to $\alpha = \mathit{float}$ by the body `count - x` (count:float), requiring no separate monomorphization pass (HM's `unify` does it automatically). Note that AIPL has no higher-rank polymorphism (rank-2 or higher); positions that take functions as arguments fall back to `any`.

Listing 3: No annotations = full inference; call sites fix the types

```
class Hello {
  var count = 0.;        // count:float from the literal 0.
  method init(n) {       // n is unannotated
    count = n;           // count:float forces n:float
    print("hi " + n);
  }
  method greet() { print("count = " + count); }
}
var h = new Hello(5);    // re-fixes init's argument type from the call site
```

4.2.1 The Two-Pass Inference Algorithm

The central challenge in applying HM to an actor language is the *mutual reference among methods* and the *sharing of fields*. When method `m` of class `C` calls another method `n` of the same class via `self.n(…)`, `m` must be checkable even if `n`'s type is undetermined. AIPL solves this in two passes.

1. **Pass 1 (pre-registration)**: assign a fresh type variable to each field in the class, and a provisional signature $\vec{\alpha} \to \beta$ (fresh variables for arguments and result) to each method, registering them in the type environment Γ.
2. **Pass 2 (body checking)**: check each method body bottom-up and solve, via `unify`, the equality constraints produced by assignments, calls and operations. Because fields are shared across methods, a use in one method may fix the type in another.

Unification in Pass 2 is standard union-find with an occurs-check; `instantiate` freshens the generalized scheme of a `let` binding, and `generalize` closes, under $\forall$, the type variables not free in the environment. In Listing 3, Pass 1 gives `init` the type $(\alpha) \to \mathit{unit}$, and Pass 2 derives $\alpha = \mathit{float}$ from the body `count = n` (with `count : float`). Furthermore, since the call site `new Hello(5)` supplies the integer literal `5`, `init`'s argument is finally re-fixed to `int`.

4.2.2 Principal Unification Rules

Unification $\mathsf{unify}(\tau_1, \tau_2)$ follows the rules below (excerpt). σ is a substitution and $\doteq$ denotes a pair being unified.

$$\begin{aligned}
\alpha \doteq \tau &\;\Rightarrow\; [\alpha \mapsto \tau] \quad (\alpha \notin \mathrm{ftv}(\tau) \text{ occurs-check}) \\
(\tau_1 \to \tau_2) \doteq (\tau_1' \to \tau_2') &\;\Rightarrow\; \mathsf{unify}(\tau_1, \tau_1');\ \mathsf{unify}(\tau_2, \tau_2') \\
\mathsf{actor}(C) \doteq \mathsf{actor}(C') &\;\Rightarrow\; C = C' \\
\mathsf{record}(R_1) \doteq \mathsf{record}(R_2) &\;\Rightarrow\; \text{unify common fields (width subtyping)} \\
\mathsf{any} \doteq \tau &\;\Rightarrow\; \text{succeed (gradual boundary; insert transient cast)}
\end{aligned}$$

The last `any` rule is the junction with gradual typing and is the locus of the soundness boundaries of §4.4.

4.3 The Second Axis: Gradual Typing

While centered on the inferred dialect, AIPL retains the annotated dialect as a *reference implementation* and provides gradual-typing features.

- **Capability effects**: `function read_url(u:string)->string !{net,fs}{…}` annotates method types with an effect row over `{ai,fs,net,mut}`. Default effects `BUILTIN_EFFECTS` are assigned to 43 builtins and inferred.
- **Linear types**: `var fh: linear int = open("x");` statically guarantees single use of a resource.

- **Ownership**: `pub var holder: string = "";` distinguishes public fields.
- **Session types**: `session_define("Solve", "solver.solve(string) ! string -> reviewer.review(…) -> …")` dynamically defines an inter-actor protocol.
- **Refinement**: predicate constraints over integers, reals and rationals are delegated to the SMT solver Z3 (detecting vacuously-false predicates at declaration time).

These exist in the inferred dialect only through the runtime, while the annotated dialect carries the static-checking reference implementation; this is the two-axis structure.

4.4 Type Soundness

Type soundness is discussed via the two propositions of progress and preservation.

Definition 1 (Progress)**.** *Execution of a well-typed program does not get stuck due to a type mismatch (e.g. an arithmetic operation on a string).*

Definition 2 (Preservation)**.** *After taking an evaluation step, a value's type remains compatible with its declared type (under a simple partial order that includes* `any`*).*

Being a gradually typed language, AIPL exhibits the typical property of being *locally sound but globally unsound*. The boundaries at which soundness may break are four: (1) inflow of `any` from unannotated to annotated regions; (2) changes of class shape via dynamic reflection (`compile` / `become` / `add_method`); (3) divergence of the recipient's signature under dynamic dispatch; and (4) rewriting an actor field via a record. For the highest-priority unsoundness gap, the `any`→`T` boundary, we introduced a **transient cast** via the `--transient` flag, inserting at the three boundaries (variable declaration, method argument, function argument) a runtime check governed by the same compatibility rule as the static check.

5 Realization: Seven Runtimes and Nine Backends

5.1 The Multi-Implementation Architecture

AIPL has **seven runtime implementations** of a single source language (Table 2). Of these, HM inference is adopted by three: OCaml, inferred Python, and the C code generator.

Table 2: AIPL's seven runtime implementations

Short name	Source	Type system
Py-A (annotated)	`python-aipl/`	Gradual typing (Phase 11–17 reference, most complete)
Py-I (inferred)	`python-aipl-inferred/`	**HM inference** (interp shared with Py-A, 899 lines)
OCaml (canonical)	`src/*.ml`	**HM inference** (`infer.ml` 484 lines, WebSocket built in)
JS-O (via OCaml)	`src/app.js` + gateway	= OCaml HM (`/api/typecheck`)
JS-B (browser)	`browser-abcl/`	flow-sensitive light inference (`typecheck.js` 308 lines)
JS-N (Node)	`node-aipl-server/`	flow-sensitive (inherited), HTTP wrap
C (`aipl2c`)	`aipl2c.ml` + C runtimes	**HM + code specialization**

5.2 Parsing Mechanism

The canonical OCaml implementation performs lexing/parsing with `lexer.mll` (ocamllex) + `parser.mly` (Menhir), feeding `infer.ml` → `eval_thread.ml`. The Python implementation uses a Lark grammar (`grammar.lark`) and passes the AST to `aipl_typeck` / `aipl_inference`. The browser/Node implementations share browser-abcl's hand-written parser.

5.3 HM-Driven Code Specialization

The core technique of the C backend (`aipl2c`) is **code specialization** using HM inference results. If `l_a, l_b` are inferred to be `int`, then instead of the `value_t` tagged-union operation `v_binop("+",l_a,l_b)` it emits the native `long l_x = (l_a + l_b);`. `value_t` is kept uniform only at actor message boundaries (`value_t args[]`), while the actor interior is specialized. This reconciles *mailbox uniformity* with *static specialization of the interior*.

5.4 Nine Code-Generation Backends

The OCaml `aipl2c` generates code from a single frontend to several backends (Table 3). A send is projected onto each target's concurrency primitive: C+pthread emits `enqueue(…)`, Pony `obj.method(args)`, Erlang `Pid ! {method,Args}`, Go `obj.mailbox <- msgT{…}`, and Prolog `thread_send_message(…)`. Pony/Go/Erlang use a two-stage initialization that separates construction from the init body.

Table 3: Nine backends from a single frontend

Flag	Target	Concurrency model
(default)	C + pthread	1:1 OS threads
`-lSDL2`	C + SDL2 GUI	1:1 (with rendering)
`--xinu`	Xinu C	bare-metal cooperative
`--python`	Python	1:1 OS threads
`--pony`	Pony	M:N lightweight
`--erlang`	Erlang/BEAM	M:N lightweight
`--go`	Go goroutine	M:N lightweight
`--prolog`	SWI-Prolog	threads
`--llvm / --openmp`	LLVM / OpenMP	hot/cold annotation, parallel for

6 Execution Environment (1): Concurrency Models and LLM Integration

6.1 Unifying Three Concurrency Models

A key demonstration in AIPL is that the same `.abcl` source can be deployed, *unmodified*, across three concurrency models.

1. **1:1 OS threads** (Python/OCaml/C): on the order of 100–1000 actors.
2. **M:N lightweight threads** (Pony/Erlang/Go codegen): on the order of 10^5–10^7 actors.
3. **Cooperative event loop** (browser-abcl/node): single-threaded `setTimeout(0)` scheduling.

Because the actor abstraction is independent of the concurrency model, one can migrate from a prototype (event loop) to high throughput (M:N) without changing the source.

6.2 LLM Governance

The `ai_call` family (`ai_call_with_system`, `ai_call_retry`, `ai_call_image`, `ai_call_priority`) is governed by the runtime through environment variables: `ABCL_AI_TOKEN_BUDGET` (`BudgetExceeded` on overflow), `ABCL_AI_MAX_CONCURRENT` (in-flight semaphore), `ABCL_AI_FALLBACK_MODELS` (fallback chain), `ABCL_AI_USAGE_FILE` (usage accounting), and `AIPL_AI_PROVIDER=mock` (offline smoke). Backends support Ollama (local), OpenAI, Gemini and Anthropic.

6.3 WebSocket and Dashboards

All seven runtimes provide WebSocket over a unified wire protocol: OCaml implements RFC 6455 by hand in `web_gateway.ml` (40 lines), Python in `aipl_websocket.py`, JS-B uses the browser native API, JS-N uses `ws`, and C uses libwebsockets. A client joins via `?sid=<group>` and receives `{"kind":"welcome","sid":…,"peers":N}`. `aipl_main.py --dashboard 8800` provides live counters, an observed traffic table and an SSE event stream, and `ABCL_PEER_DASHBOARDS` aggregates multiple workers into a TOTAL row.

7 Execution Environment (2): Distributed Runtime and On-Device Distribution

7.1 Distributed Runtime DR-1..13

Enabled by `AIPL_DIST_ENABLE=1`, the distributed runtime layer (`aipl_dist`) provides, opt-in, the governance features needed for operating AI agents (Table 4).

Table 4: Distributed runtime features DR-1..13

ID	Feature	Summary
DR-1	env-var routing	`AIPL_ROUTE`, actor-name→tag table
DR-2	structured log	ND-JSON, one line per event
DR-3	token-budget coordination	60-second sliding window (RPM/TPM)
DR-4	checkpoint/resume	atomic save/restore
DR-5	quarantine & skip	automatic quarantine (TTL)
DR-6	quorum replication	parallel multi-provider
DR-7	subtree quarantine	OTP-style blast-radius limiting
DR-8	spawn-tree tracking	TLS-auto parent
DR-9	runtime hooks	interp/actor/call_ai (opt-in)
DR-10	CRDT state	G-Counter/OR-Set/LWW-Register (15 prims)
DR-11	saga compensation	`saga{step{}compensate{}}` (LIFO)
DR-12	multi-region failover	per-region route + chain walk
DR-13	auto-scaling pool	hysteresis-driven (`pool_create` etc.)

7.2 HTTP/JSON Distribution and Remote Send

Remote send is realized over an HTTP/JSON wire protocol: `POST /api/json/send` (fire-and-forget) and `POST /api/json/call` (synchronous reply). Python provides `remote_call` / `remote_now` / `remote_future` and `web_listen(port)` / `web_expose` / `serve_forever`. Setting `ABCL_REMOTE_SECRET` makes an HMAC-SHA256 signature (`X-ABCL-Sig`) mandatory, wire-compatible across Python/OCaml/C (OCaml uses a pure `hmac_sha256.ml`). This enables coordination across three nodes with different providers (coordinator / solver / verifier).

7.3 On-Device Distribution: Mac × Raspberry Pi (Xinu)

AIPL further demonstrates on-device distribution spanning host actors on a Mac and AIPL actors on the bare-metal OS **Xinu** running on a Raspberry Pi. The two are connected by a line-oriented RPC over UART/TCP (opcodes: PING/LIST/SEND/QUERY/LOAD/COMPILE/RUN/SPAWN), and the inferred Python translates this transparently into sends via the `uart1://` scheme. As a representative example, the "distributed dining philosophers" was composed of 3 Mac nodes + 2 Xinu nodes, with the 5 forks turned into Xinu actors, achieving mutual-exclusion synchronization across the boundary on real hardware.

7.4 The Evolutionary Language-Design Pipeline

AIPL's next-generation features (CE-1..13 for inference enhancement and DR-1..13 for distribution) were implemented as the result of a **MAP-Elites** genetic algorithm in `aice-evolution-v2` that, taking a language specification (`.aice`) as input, performs cross-task evaluation and LLM-directed mutation to empirically derive "universal axes of language design." For example, structured concurrency (`scope{future…}`) was turned into a language feature based on a prediction in which the search pinned `concurrency=structured`, `ownership=linear` and `type=dependent` as top axes. The pipeline consists of three stages, `.aice` → `.ga.json` → `.abcl`. Table 5 shows the predicted axes the search derived and reflected into the implementation.

Table 5: Language axes derived by the evolutionary search and their implementation

Prediction (axis)	Winning value	Implementation
type_safety	dependent	refinement + Z3 (CE-6/CE-9)
effect	capability	capability effect rows (CE-10/CE-11)
concurrency	csp_channels	`channel[T]` (CE family / Phase 13)
ownership	linear	linear types `linear`
state	symbol_owned	actor-closed fields
concurrency (again)	structured	`scope{future…}`

The winning individuals (balanced / hang-resilience / Erlang-OTP-like) were selected from a MAP-Elites search of 8 seeds × 30 generations × 2 runs (about 24 minutes each, over 1400 LLM calls). These became the implementation basis for DR-1..13.

8 Case Studies

This section shows, through two case studies, how AIPL's design crystallizes in real applications.

8.1 An LLM Multi-Agent Pipeline

A configuration in which planner, solver and reviewer agents cooperate to solve a problem can be written concisely with the three send forms and `ai_call` (Listing 4). Future-form sends overlap the LLM calls of the solver and reviewer, and `scope` bundles the lifetimes of the child tasks.

Listing 4: planner→solver→reviewer cooperation

```
class Solver   { method solve(q)      { reply(ai_call(2, "Solve: " + q)); } }
class Reviewer { method review(q, a)  { reply(ai_call(2, "Grade:" + q + a)); } }
class Planner {
  var s = new Solver();
  var r = new Reviewer();
  method run(q) {
    scope {
      var fa = future s.solve(q);     // solver's LLM call
      var a  = await(fa);
      var fr = future r.review(q, a); // reviewer's LLM call
      reply(await(fr));
    }                                 // unfinished futures auto-joined on scope exit
  }
}
var p = new Planner();
print(now p.run("integral of x^2"));
```

The token budget (`ABCL_AI_TOKEN_BUDGET`), concurrency (`ABCL_AI_MAX_CONCURRENT`), quorum replication (DR-6) and fallback (`ABCL_AI_FALLBACK_MODELS`) are all governed by environment variables and

the runtime without changing this source.

8.2 Mac × Xinu On-Device Distribution: Dining Philosophers

We ran the classic dining-philosophers problem in a mixed topology that places the 5 forks as Xinu actors on a Raspberry Pi and the philosophers as actors on a Mac. The inferred Python on the Mac side translates sends to the fork actors transparently into line-oriented RPC (QUERY/SEND) under the `uart1://` scheme, achieving mutual-exclusion synchronization across the boundary on real hardware. The fact that the same philosopher logic can be deployed *unmodified* into the three forms (a) single-process 1:1 threads, (b) M:N via Go-goroutine codegen, and (c) Mac+Xinu on-device distribution, concretely substantiates the concurrency-model independence of §6.

9 Evaluation

9.1 Cross-Validation of Inference

AIPL's principal empirical claim is that the three HM-equipped implementations (OCaml / inferred Python / C code generator) produce *identical inferred types* for the same `.abcl` source. A representative example follows (identical across all three):

```
Hello:   count : float;  init : (int) -> unit;
         greet : () -> unit;  inc : () -> unit
Counter: count : float;  inc : () -> unit;
         dec : (float) -> unit   <- monomorphized via call site
```

9.2 Feature Parity and Testing

The next-generation features CE-1..13 + DR-1..13 are confirmed to have feature parity across the main runtimes. At the completion of Phase O, distributed DR-1..9 and inference CE-1..9 were each 9/9 across the Py-A / OCaml / JS-O implementations, totaling 18/18 (about 1305 LOC of implementation). JS-B / JS-N also closed out at 26/26, including CE-10/CE-12/CE-13/DR-11. Table 6 summarizes the smoke tests.

Table 6: Smoke-test summary (representative figures)

Item	Result
OCaml runtime	52/52
Browser (syntax/parse)	7/7 + 4/4
Python (`.abcl` / legacy `.aipl`)	33/33 + 21/21
Distributed cross-language (3-node mock)	8/8
Self-hosting tower (10 levels, 41 samples)	47/47
`aipl_dist` unit	27/27
Distributed MVP demos (8 features × 3)	24/24
next-gen / self-host assertions	103 all PASS
Total	**213+ tests/demos green**

9.3 Self-Hosting

The `aipl-self-host/` tower, which describes AIPL in AIPL itself, consists of 10 levels (Level A–C-3 plus the integrated Level Z), and all 47/47 smoke tests are green. Level Z is an integrated full self-host using an 18-line Python bootstrap and a CE-11 capability-checked `io_bridge.abcl`, applying each level's checker to itself to demonstrate phase monotonicity.

9.4 Implementation Size

The sizes of the principal components are: OCaml HM ≈985 lines (`infer.ml` 484 + `types.ml` + `typing_env.ml`), inferred Python 899 lines, `aipl_inference.py` ∼1255 lines, `aipl_dist.py` 591 lines, `typecheck.js` 308 lines, WebSocket 40 lines, the C+SDL2 runtime 1178 lines, and the self-host with 9 modules ∼1500 lines.

10 Discussion and Limitations

Limits of soundness. As stated in §4.4, AIPL is a gradually typed language: it is locally sound for annotated regions, but global soundness does not hold at boundaries with unannotated regions or dynamic reflection. The transient cast plugs the highest-priority gap, but the runtime type cast is limited and constitutes a weak guarantee that falls short of full transient soundness. Soundness guarantees for programs containing dynamic reflection (`become` / `compile`) are future work.

Expressiveness of inference. The current HM lacks higher-rank polymorphism (rank-2 or higher), and positions taking functions as arguments fall back to `any`. Also, narrowing is limited to simple conditions; narrowing over `&&` chains and nested conditions is unsupported.

Version drift. The figures in this paper are integrated from several reports of different periods, with version drift in the number of runtimes (7 vs 6, depending on how Py-A is counted) and the number of self-hosting levels (9/37 vs 10/47). This paper adopts the latest feature matrix (10 levels, 47/47, 7 runtimes).

Future work. (i) Strengthening soundness guarantees for programs with dynamic reflection; (ii) higher-rank polymorphism and full flow-sensitive narrowing; (iii) large-scale ($>10^6$ actors) on-device evaluation on the M:N backends; and (iv) quantitative validation of language-feature prediction via the evolutionary pipeline.

11 Conclusion

This paper presented, in a single coherent account, the design, implementation and execution environment of AIPL, a typed AI agent language that gives a first-class linguistic expression to the "cooperating multiple agents" that form the natural architecture of LLM-driven AI systems. AIPL layers HM type inference and gradual-typing features, together with first-class AI integration via `ai_call`, onto ABCL/1's three send forms; it drives seven runtimes, nine backends and three concurrency models from one source, and realizes on-device distribution spanning a Mac and a Raspberry Pi (Xinu) over HTTP/JSON and UART/TCP. We empirically showed that the three HM-equipped implementations produce identical inferred types for the same source, and that more than 213 tests/demos are green. AIPL offers one attainment of a "scripting layer for an AI-OS" that bridges the classical ideal of actor languages and the modern operation of AI agents.

References

[1] C. Hewitt, P. Bishop, R. Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. *IJCAI*, 1973.

[2] A. Yonezawa, J.-P. Briot, E. Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA*, 1986.

[3] J. R. Hindley. The Principal Type-Scheme of an Object in Combinatory Logic. *Trans. AMS*, 1969.

[4] R. Milner. A Theory of Type Polymorphism in Programming. *J. Computer and System Sciences*, 1978.

[5] L. Damas, R. Milner. Principal Type-Schemes for Functional Programs. *POPL*, 1982.

[6] J. G. Siek, W. Taha. Gradual Typing for Functional Languages. *Scheme Workshop*, 2006.

[7] A. Mettler, D. Wagner, T. Close. Joe-E: A Security-Oriented Subset of Java. *NDSS*, 2010.

[8] P. Wadler. Linear Types Can Change the World! *Programming Concepts and Methods*, 1990.

[9] K. Honda, V. T. Vasconcelos, M. Kubo. Language Primitives and Type Discipline for Structured Communication-Based Programming. *ESOP*, 1998.

[10] R. Pressler et al. JEP 453: Structured Concurrency (Java Project Loom). 2023.

[11] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-Free Replicated Data Types. *SSS*, 2011.

[12] J.-B. Mouret, J. Clune. Illuminating Search Spaces by Mapping Elites. arXiv:1504.04909, 2015.